

A SIMPLE, MINIMAL DSL FOR ASC

SMALL MODEL VS COMPILER VS CONSTRAINED DECODE

Date: 2026-08-22

Status: conversation capture / design instrument (not a spec, not an implementation plan)

Context: README § “ASC domain-specific language : *DSL* syntax” is the **only source of truth** for this syntax (the README is the only doc written entirely by the author). Revival v4 (Projet Complexe 2026 Revival (v4) - ASC, Projet Complexe and Projet Complexe ASC.md) for the agent / tool-boundary cut.

Origin: Cursor chat, 2026-08-22. Written out in full so the thread can be resumed from this file.

This note exists because the README DSL looks small enough to “just teach a model,” and that idea is easy to over-build — or to under-build, by skipping the one cheap piece that makes everything else safe.

The useful question is not only “is a small model possible?” It is:

Which job is neural (authoring DSL from natural language) and which job is symbolic (parsing, compiling, rejecting invalid or dangerous strings) — and which of those belongs in ASC core, which is a Fallback for tiny local models, and which should wait until the README’s own remaining TODOs are closed?

0. WHAT THE README DSL ACTUALLY IS

The syntax in the README is a compact, **filename-safe** combinator language. It is meant to address argv the way `make` addresses entry points, and to appear in YAML `validate:` fields and in hook filenames such as:

```
*/$subject/transcribe-file(v-input_file_path).pre-index.hook.sh
```

The rewrite checklist still has **Stabilize DSL** unchecked. That means “write the PEG / runtime that matches this README,” not “the README is one proposal among others.” The README is the language. A few lines inside it are still marked `TODO [wip]` (see §0.2); those are holes *in* the SoT, to be closed *in* the README.

0.1 SURFACE

Entry points (as in `make`):

- `start = make start = asc/instance/start.sh`
- `service-rebuild = make service-rebuild = asc/extensions/compose/service/rebuild.sh`

Positional arguments:

- `@ = $@` (all arguments forwarded as-is)
- `p1 = $1, p2 = $2, ...`

Boolean options (shrink all `--` to `-` in prefixed syntax):

- `b-online = --online`
- `b-y = -y`
- `b-@ =` all boolean options forwarded (TODO [wip] YAGNI?)

Named options:

- `o-max=4 = --max=4 or --max 4 or -m 4` (TODO [wip] how to distinguish)
- `o-@ =` all named options forwarded (TODO [wip] YAGNI?)

Variables (any bash var in hook calling scope), `v-` prefix:

- `v-input_file_path = $input_file_path` in a `hook()` calling scope

Functions (whitelisted bash functions), `[]` enclosure:

- `[echo(a)] = echo "$@"`
- `[f_db_clear(foobar)] = f_db_clear 'foobar'`
- `[f_db_clear(v-DB_NAME)] = f_db_clear "$DB_NAME"`
- Nested / subshell TODO [wip]: `[f_db_clear(slug(p1))]` → `f_db_clear "$(slug 'foobar')"` ?

Chaining:

- `[echo(v-baz)];echo(foobar) → echo "$baz" ; echo 'foobar'`
- `[echo(v-baz)];;echo(foobar) → echo "$baz" && echo 'foobar'`

Parallel:

- `[echo(v-baz)]-;-echo(foobar) → echo "$baz" & echo 'foobar' ; wait`

Piping:

- `[echo(v-baz)]+grep(foobar) → echo "$baz" | grep 'foobar'`

Conditional:

- `[echo(v-baz)]+[exit(1)] → echo "$baz" || exit 1`

Redirect:

- `[echo(v-baz)]--v-output_file_path → echo "$baz" > "$output_file_path"`

- `[echo(v-baz)]---v-output_file_path → echo "$baz" >> "$output_file_path"`

Loops:

- `[[echo(v-item)]v-foobar_arr] → for item in "${foobar_arr[@]"; do echo "$item"; done`
- `[[echo(v-item)]myfunc] → while read -r item; do echo "$item"; done < <(myfunc)`

“Normal” example:

```
transcribe-file(path/to/file.mp4)
```

→ `asc/extensions/transcription/transcribe/file.sh path/to/file.mp4`

YAML example (foobar.entity.yml, field `toto` must be slug or snake):

```
required:
  field:
    toto:
      validate: test-in(p1,slug(p1),snake(p1))
```

Any DSL starting with `test-*` is meant to compile to `[[ */test/*.sh ]] || exit 1.`

This is a **tiny combinator language**: entry points, a handful of prefixes (`p1` / `b-` / `o-` / `v-`), and a handful of operators (`;` `;;` `-;` `+` `++` `--` `---`, loops). That smallness is what makes both a parser and a small model *possible*. It is also what makes a neural interpreter the wrong default.

0.2 OPEN LINES STILL INSIDE THE README

Not a second syntax. These are the README’s own unfinished edges:

- `b-@` / `o-@` — YAGNI?
- `o-max-4` — `--max=4` vs `--max 4` vs `-m 4`
- nested calls — subshell or not (`([f_db_clear(slug(p1))])`)
- when YAML `validate:` runs (during entity validation?)

A PEG should encode the README as written. Those TODOs stay explicit holes until they are decided **in the README**. There is no parser yet; “Stabilize DSL” is the work of matching this surface in code.

1. IS A SMALL MODEL POSSIBLE?

Yes. The language is tiny, regular, and combinatorial. A small model can emit it.

Revival v4 already named that path, and already limited it.

1.1 WHAT V4 ALREADY DECIDED

v4 §0.4 **Refuse:**

teaching the model a private DSL as its native function-call language

v4 §1.1:

DSL is a compact, filename-safe encoding of argv. It is for *addressing and validation*, not for chatting with a model.

v4 §1.3 table, “DSL instead of JSON/Python”:

No at the model boundary. Yes as ASC addressing. Compile YAML+DSL → JSON Schema / MCP tool list / TypeBox. Never the reverse as source of truth.

v4 §4.3, the precise split:

Teaching a 7B model to emit `o-max-4` as a function call is a **fine-tune / constrained-decode research project**, not a revival decision. Even then, the canonical description remains YAML.

Exception (small): for *tiny* local models with no native tool calling, `post_llm` can parse a **constrained** mini-format (even DSL-like) *as a Fallback Implementation* of “the model requested a tool.” Do not make that the protocol you show Claude.

v4 §5.3 mutually exclusive:

JSON Schema as *projection* vs DSL as the provider’s function-call language for frontier models — fights the ecosystem; split the Fallback for tiny local models only.

So: technically possible; classified as **research / Fallback**; refused as identity; YAML `able` stays source of truth for contracts; JSON Schema is the projection frontier models already know how to emit. The **DSL text** itself is specified only in the README.

1.2 TWO CONSTRAINTS BEFORE ANY TRAINING IS WORTH IT

1. **Close or encode the README’s** `[wip]` **holes** (§0.2) before baking them into weights. A LoRA that guesses `o-max-4`’s meaning is inventing SoT.
2. **“Learn the syntax” is the wrong job for weights.** Syntax is a parser (PEG / GBNF / recursive descent) of the README. The useful model job is **intent → valid DSL, or repair, under** that grammar.

If the goal is “a small model that speaks ASC,” the cheap order is:

1. Write a PEG that matches the README surface (and leaves \$0.2 as explicit open productions until the README closes them).
2. Generate synthetic pairs from it (enumerate valid trees → captions).
3. **Constrained decode** (llama.cpp GBNF / Outlines) so a 1–7B local model *cannot* emit invalid DSL.
4. LoRA only later, and only on that grammar, if evals show few-shot + constraints are not enough.

Fine-tune as v1 is what v4 / Yu & Yao refused: it is a *project*, not a toggle. Raschka’s “train a local reasoner” door stays **late**, not day one.

1.3 FIRST FORK OFFERED IN THE CONVERSATION (A / B / C)

What “learn that syntax” could mean:

- **A.** A formal grammar + parser (no neural net) so ASC can compile DSL to bash.
- **B.** Constrained decode: a small local LLM may only emit valid DSL (v4 Fallback for models without tool calling).
- **C.** Fine-tune / LoRA a tiny model that authors DSL from natural language.

2. “THE DSL IS SO SIMPLE IT DOES NOT WARRANT A COMPILER” → THEREFORE C?

C is the right **authoring** idea. It is the wrong **replacement** for a compiler.

2.1 A COMPILER FOR THIS DSL IS NOT A COMPILER PROJECT

The “too simple for a compiler” read is half-right: nobody wants LLVM.
A compiler for this DSL is a short recursive-descent or PEG: tokenize prefixes, nested `()` / `[]`, then `;` `;;` `+` `++`. On the order of **100–300 lines**. That is cheaper than gathering LoRA data, and it is the only thing that can **reject** `rm -rf` dressed up as DSL.
Revival v4 needs that gate: YAML `able` is canonical for contracts; the model never invents the tool surface; the model never sees `make hook`.

2.2 WHAT A TINY MODEL STILL CANNOT DO WITHOUT A PARSER

A tiny model that “knows the syntax” still emits **strings**. Without a parser you cannot:

- put DSL in `validate:` fields and trust the result
- generate or match hook filenames
- enforce an allowlist of entry points
- compile to bash deterministically
- produce training data that is *known* valid
- score a model (exact match after parse, not BLEU on the string)

LoRA without a parser is training a model whose mistakes execute. LoRA that fills README `[wip]` holes in the weights is a second, unofficial SoT.
The parser is also how you *get* C’s dataset: enumerate valid trees → English / French / Portuguese captions → LoRA pairs. And how you *score* C: parse, then exact-match the AST (or the compiled bash), not token overlap.

2.3 SO: DO C FOR NL → DSL, KEEP A TINY PARSER AS THE COMPILER

Skip LoRA at first. A 1–3B instruct model + a GBNF/Outlines grammar that **is the same** PEG already forbids invalid syntax. LoRA only if that fails an eval set.

2.4 THREE SIZES

	What it is	Cost	When it wins
1. PEG only	DSL → bash; humans / YAML write DSL	An evening once the PEG matches the README	Hook names, <code>validate:</code> , make. Needed anyway.
2. PEG + constrained decode (recommended path toward “small model”)	Existing tiny LLM may only emit valid DSL; parser executes	Grammar + a few dozen gold NL→DSL pairs	Local models with no JSON tool calling (v4 Fallback). No training run.
3. LoRA on top of 2	Specialist 0.5–1B authors DSL from fr/en/pt	Dataset + evals + PEG that matches the README	Only if 2’s error rate on combinators (pipes, loops) is actually bad.

Do not start at 3. Do not skip 1.

Size 2 is C’s *benefit* (natural language in, valid DSL out) without C’s *cost* (a training run, a specialist checkpoint, a second identity next to JSON tool calling).
Size 3 is C done properly, and only after 1 and 2 exist.

3. WHY “SYNTAX LEARNING” AND “COMPILATION” ARE DIFFERENT JOBS

Job	Right tool	Failure mode if you use the other
DSL → bash / argv / hook path	PEG, recursive descent, GBNF used as <i>spec</i> of the README	Model hallucinates operators; unsafe strings run
NL / intent → DSL	Small instruct model, optionally LoRA	Parser cannot guess what the human meant
Model → tool call (frontier)	JSON Schema projection of <code>able.yml</code>	Fighting every provider SDK; v4 refuse
Model → tool call (tiny local, no native tools)	Constrained mini-format, possibly DSL-like, parsed in <code>post_llm</code>	Making that the protocol you show Claude

A neural net that “learns the syntax” is trying to be a **parser with extra steps**. For a language this small, that is strictly worse than PEG: slower, non-deterministic, un-auditable, and useless for filename-safe hooks.

A neural net that **authors** DSL from natural language is a **translator**. That job is real. It still needs the parser on the output side, the way a programmer still needs `bash -n` or a compiler after Copilot.

Constrained decoding sits in the middle: the weights propose; the grammar forbids illegal tokens *while generating*; the parser still type-checks / allowlists / compiles. Same PEG in three roles (spec, decode mask, runtime compiler). That is the whole trick. You do not need three implementations of the language if the GBNF is generated from the PEG (or is the PEG).

4. ALIGNMENT WITH REVIVAL V4 (DO NOT REOPEN CLOSED DOORS)

Keep:

- README as the **only** specification of DSL text.
- YAML `entity` / `able` as source of truth for **contracts**.
- JSON Schema / MCP `tools/list` / `TypeBox` as **projections** of those contracts.
- DSL as ASC **addressing and validation** (argv, filenames, `validate:`).
- `pre_llm` / `post_llm` as the harness wrap.
- Tools as allowlisted entry points.
- Tiny local models: constrained mini-format as **Fallback**, not as the Claude-facing protocol.
- Routing / cascade among independently trained models (`2603.04445`): a 7B that emits JSON tool calls and a 0.5B that emits DSL would be two Technologies, not two sources of truth.

Refuse:

- Teaching frontier models a private DSL as native function-call language.
- Fine-tune as v1 / as identity.
- Python/TS `@tool` registries that bypass YAML.
- A LoRA (or any other doc) as a second specification of the syntax.

Do not close:

- A later LoRA on the README grammar for a personal termbase / local authoring model (Yu & Yao: LoRA is a later door).
- A later local reasoner (Raschka) — unrelated to *this* DSL, do not conflate.

The ASC-shaped gap in v4 \$4.2 remains small: `llm` entry point + hooks; `able.yml` → JSON Schema; dispatcher from tool name → allowlisted entry point; traces. A specialist DSL-authoring model is **not** that gap. It is an optional Implementation of “the tiny local model requested a tool,” behind `post_llm`.

5. PRACTICAL ORDER OF WORK (WHEN RESUMED)

Not an implementation plan. A resume checklist.

1. Treat the README DSL section as the grammar. Close or explicitly keep the `[wip]` lines in the README (`b-@` / `o-@`, `o-max-4` form, nested subshell, when `validate:` runs).
2. Write a PEG (or equivalent) + gold examples taken from that README section, including reject cases (`rm`, unquoted redirects, unknown entry points).
3. Compile DSL → bash / argv; use it for `validate:` and for hook-stem checks. This is size 1. It is what “Stabilize DSL” on the rewrite checklist means.
4. Generate GBNF from the same grammar. Few-shot a local 1–3B. Eval on held-out NL→DSL pairs. This is size 2.
5. Only if size 2 fails on combinators: synthetic dataset from the parser, then LoRA (size 3). Eval = AST exact match + allowlist violations = 0.

Until step 1’s `[wip]` holes are either closed in the README or marked as out-of-scope for the parser, a LoRA that fills them is inventing SoT.

6. OPEN QUESTION (RETURN HERE)

Most “English → ASC” traffic is **one pivot + args** (`transcribe-file` + a path). That is ordinary tool calling. JSON Schema already does it; the model does not need `+ ; ; - ; -`.

The full combinator language (pipes, loops, filename hooks) is a **small programming language**. Teaching a 0.5B to write programs is real, and eval-heavy. That is when LoRA might eventually pay.

Which target should the tiny model author?

- **Single call:** natural language → one allowlisted entry point + args (JSON is enough; DSL is optional sugar).
- **Full combinators:** natural language → pipes, `;;`, loops, hook-stem DSL.

Until that is answered, default to size 1 (parser) for ASC core, and treat size 2 as the only model experiment worth a spike — and only as v4’s Fallback for tiny local models, not as the authoring UX for Projet Complexe.

7. POINTERS

- README § DSL (SoT): `/home/paul/Documents/asc/README.md`, from “ASC domain-specific language : *DSL* syntax”.
- Revival v4, especially §0.4 refuse list, §1.1–1.3, §4.3 “DSL instead of JSON”, §5.3 mutually exclusive row on JSON Schema vs DSL-at-the-provider.
- Related: Yu & Yao (language system, LoRA as later door); Raschka (train a reasoner: late door, not this DSL); Moslem & Kelleher `2603.04445` (routing/cascade, not a specialist syntax model as identity).